

Final Year Design Project

AI-Powered Disease Recognition System



By

Muhammad Hassan 2716

Muhammad Rohan Ashraf 2720

Under the supervision of

Prof. Hafiz Shahzad & Muhammad Fahim

Bachelor of Science in **Information Technology** (2022-2026)

Govt. Graduate College Gulberg For Boys, LAHORE.

AI-Powered Disease Recognition System

A project presented to

University of the Punjab, Lahore

**In partial fulfilment of the
requirement for the degree of**

***Bachelors of Science in Information Technology
(2022-2026)***

By

Muhammad Hassan 082153

Muhammad Rohan Ashraf 082155

**FACULTY OF COMPUTING &
INFORMATION TECHNOLOGY (FCIT),
UNIVERSITY OF THE PUNJAB, LAHORE**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: -----

Muhammad Hassan (2716)

Signature: -----

M. Rohan Ashraf (2720)

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSIT “AI disease recognition” was developed by

Muhammad Hassan (082153) & Muhammad Rohan Ashraf (082155)

under the supervision of “ **Prof. Hafiz Shahzad** ” in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in **Information Technology**

Signature: -----

FYDP Supervisor:

Signatures (Faculty Advisory Committee (FAC))

Signatures		
Name	Prof. Hafiz Shahzad	Prof. Muhammad Fahim
	FAC1	FAC2

Signature: -----

Head of FYDP Coordination Office:

Signature:-----

Dated: _____

Chairperson, Department of **Information Technology**

Executive Summary

The proposed project, **AI Medical Assist**, is an advanced AI-based healthcare system designed to predict and monitor **Chronic Kidney Disease (CKD)** and **Diabetes Mellitus (DM)** using machine learning models. The trained models are deployed through the **Django REST Framework** for real-time prediction and analysis. A **Gemini conversational AI** interface enables users to interact naturally, receiving personalized explanations and health guidance based on model results. Additionally, the system integrates the **Google Maps API** to locate nearby doctors or clinics for high-risk users. By combining predictive analytics, conversational intelligence, and geolocation support, AI Medical Assist offers a comprehensive and user-friendly approach to preventive healthcare.

Keywords: **Predictive Healthcare , Conversational AI , Disease Risk Prediction**

Acknowledgement

The acknowledgment section is an opportunity to express gratitude and appreciation to individuals and organizations who have contributed to the completion of your Final Year Project (FYP).

Signature: -----

Muhammad Hassan (2716)

Signature: -----

Muhammad Rohan ashraf (2720)

Abbreviations

Table of all the abbreviations and acronyms used in your FYP along with their respective brief description. The abbreviation list must be ordered alphabetically.

BL	Backlog list : List of the requirements under consideration
FCIT	Faculty of Computing and Information Technology

Table of Contents

1.	<i>Introduction.....</i>	<i>14</i>
1.1	Problem Statement	14
1.2	Problem Solution.....	14
1.3	Objectives of the Proposed System.....	14
1.4	Scope	16
1.5	System Components.....	16
1.5.1	Module 1: Module Name.....	16
1.6	Related System Analysis/Literature Review.....	16
1.7	Vision Statement	17
1.8	System Limitations and Constraints.....	17
1.9	Tools and Technologies	17
1.10	Project Deliverables.....	18
1.11	Project Planning.....	18
1.12	Summary.....	18
2.	<i>Analysis.....</i>	<i>20</i>
2.1	User classes and characteristics.....	20
2.2	Requirement Identifying Technique.....	20
2.3	Functional Requirements.....	20
2.3.1	Functional Requirement X	20
2.4	Non-Functional Requirements	21
2.4.1	Reliability.....	21
2.4.1	Usability	21
2.4.2	Performance.....	22
2.4.3	Security	22
2.5	External Interface Requirements.....	22
2.5.1	User Interfaces Requirements	22
2.5.2	Software interfaces.....	23
2.5.3	Hardware interfaces.....	23
2.5.4	Communications interfaces	23
2.6	Summary.....	23
3.	<i>System Design</i>	<i>25</i>
3.1	Design considerations.....	25
3.2	Design Models.....	25
3.3	Architectural Design.....	25

3.4	Data Design	25
3.4.1	Data Dictionary.....	26
3.5	User Interface Design.....	26
3.3.1	Screen Images	26
3.3.2	Screen Objects and Actions.....	26
3.4	Behavioural Model	26
3.5	Design Decisions.....	27
3.6	Summary.....	27
4.	<i>Implementation</i>	29
4.1	Algorithm.....	29
4.2	External APIs/SDKs	29
4.3	Code Repository.....	30
4.3.1	Metrics of the Git Repository:.....	30
4.4	Summary.....	30
5.	<i>Introduction.....</i>	32
5.1	Unit Testing (UT)	32
5.2	Functional Testing (FT).....	32
5.3	Integration Testing (IT).....	32
5.4	Performance Testing (PT).....	32
5.5	Summary.....	33
6.	<i>Introduction.....</i>	35
6.1	Conversion Method	35
6.2	Deployment.....	35
6.2.1	Data Conversion.....	35
6.2.2	Training	35
6.3	Post Deployment Testing	35
6.4	Challenges	36
6.5	Summary.....	36
7.	<i>Introduction.....</i>	38
7.1	Evaluation	38
7.2	Traceability Matrix.....	38
7.3	Conclusion.....	38
	7.4 Future Work.....	39
	References.....	40

<i>Appendix-A Use Case Description(Fully Dressed Format).....</i>	<i>42</i>
<i>Appendix-B General Coding Standards & Guidelines</i>	<i>44</i>
<i>Appendix-C Application Prototype.....</i>	<i>46</i>

List of Figures

List of Tables

Chapter 1

Introduction

1. Introduction

This chapter provides an overview of the **AI Medical Assist** project. It outlines the problem being addressed, the proposed solution, project objectives, and the technological foundation of the system. The chapter also defines the project's scope, architecture, components, and deliverables. By combining **machine learning**, **AI-driven conversation**, and **geolocation services**, this system aims to offer an end-to-end, intelligent, and user-friendly health prediction platform for early detection and advisory support in **Diabetes Mellitus (DM)** and **Chronic Kidney Disease (CKD)**.

1.1 Problem Statement

Chronic diseases such as **Diabetes Mellitus** and **Chronic Kidney Disease (CKD)** represent some of the most widespread health challenges globally, leading to severe complications and increased mortality if not detected early. In developing regions, patients often lack access to timely laboratory diagnostics, personalized health monitoring, and professional consultation.

Existing medical applications primarily focus on **record-keeping or basic symptom checking**, offering little predictive or advisory intelligence. Moreover, traditional diagnosis relies heavily on manual interpretation of lab results by medical professionals, which can delay early intervention.

The **AI Medical Assist** project addresses this gap by leveraging **machine learning** for accurate disease stage prediction and integrating **conversational AI (Gemini)** for explainable and human-like health advice. The problem being solved is **the lack of accessible, AI-powered early disease detection and personalized recommendation systems** that combine real-time prediction, conversational interpretation, and nearby doctor connectivity.

1.2 Problem Solution

The proposed solution, **AI Medical Assist**, is a mobile-first AI-driven health assistant that predicts the likelihood and stage of **Diabetes** and **Chronic Kidney Disease** based on patient inputs such as laboratory results and clinical indicators.

Using **trained ML models (XGBoost, LightGBM, and scikit-learn pipelines)** hosted on a **Django REST Framework backend**, the system provides:

- **Real-time risk prediction** with confidence scoring.
- **Stage classification** (e.g., Normal / Prediabetes / Diabetes; CKD Stage 1–5).
- **Explainability using SHAP**, identifying the top contributing health factors.

- **Natural language explanations** via the **Gemini conversational API**, mimicking a virtual doctor.
- **Google Maps integration** for nearby doctor or hospital recommendations.

By combining AI prediction, explainability, and conversational assistance, AI Medical Assist enables users to understand their health risks, receive guidance, and take preventive actions in a single integrated platform.

1.3 Objectives of the Proposed System

The main objectives of **AI Medical Assist** are both **technical** and **functional**, ensuring real-world applicability, scalability, and user empowerment.

Textual Objectives

- To develop a dual-disease prediction system using **machine learning models** trained on medical datasets (PIMA for Diabetes, UCI CKD for Kidney Disease).
- To classify disease stages with **quantifiable confidence levels** and explainable parameters.
- To provide **AI-based conversational insights** using Gemini API without exposing personal health information (PHI).
- To integrate **Google Maps API** for location-based doctor recommendations.
- To ensure **secure and personalized** user experience through JWT-based authentication and PostgreSQL-backed history.
- To establish **modular, scalable, and privacy-compliant architecture** suitable for future EHR integration.

Objective ID	Objective Description	Measurement Criteria	Timeline
--------------	-----------------------	----------------------	----------

OBJ-1	Train and deploy ML models for Diabetes & CKD risk prediction	Model accuracy $\geq$ 85%	Week 4–8
OBJ-2	Integrate SHAP-based explainability for feature importance	Top-5 features visualized per prediction	Week 8–10
OBJ-3	Implement Gemini conversational API	Human-like AI explanations generated	Week 10–12
OBJ-4	Develop Flutter-based mobile app with secure JWT auth	Functional login/signup & dashboard	Week 6–9
OBJ-5	Integrate Google Maps API for doctor lookup	Display nearby medical facilities	Week 11–13
OBJ-6	Conduct end-to-end testing and report generation	Successful test cases $\geq$ 90% pass rate	Week 13–15

1.4 Scope

The project focuses on creating a **mobile-first health monitoring ecosystem** powered by AI, enabling non-specialist users to predict, understand, and act on health risks.

In Scope:

- Dual-disease prediction (Diabetes, CKD)
- Real-time inference using Django REST APIs
- AI-based chat explanation using Gemini
- Nearby doctor search using Google Maps API
- Secure user authentication and result history

Out of Scope:

- Real physician consultation or prescription generation
 - Cloud-hosted medical record storage beyond PostgreSQL prototype
 - Diagnosis of diseases beyond Diabetes and CKD
- 1.

1.5 System Components

The system is composed of interlinked modules that form a cohesive full-stack architecture.

Component	Technology Stack	Purpose
Flutter Frontend	Flutter, Dart,	User interaction and input collection
Django REST Backend	Python, DRF, JWT	Core logic, authentication, and API orchestration
ML Engine	scikit-learn, XGBoost, SHAP	Disease prediction and explainability
WebSocket Server	Django Channels, Redis	Real-time communication with Gemini
Database	PostgreSQL	Secure user, chat, and prediction data storage
Maps Service	Google Maps API	Nearby doctor/hospital location integration

1.5.1 Module 1: Prediction Module

Features:

- Accepts user's lab data inputs (e.g., glucose, creatinine, blood pressure).

- Runs ML models for Diabetes and CKD.
- Generates risk score, disease stage, confidence level.
- Returns explainable SHAP feature impacts and health recommendations.

1.6 Related System Analysis/Literature Review

Briefly provide an analysis of the related system which may help you to specify the contribution of the proposed project. This includes the characteristics of the existing/similar systems related to your proposed project. Don't use more than 4 sentences for explaining a single system/application. Highlight its characteristics including the features lacking in the existing systems that may be present in your system.

Table 1- 1 Related System Analysis with proposed project solution

Existing System	Weaknesses	Proposed System Advantage
Ada Health / Symptomate	Focused on symptom-based analysis only	AI Medical Assist uses lab data for higher medical precision
MySugr	Diabetes-only focus	Supports both Diabetes & CKD with predictive staging
KidneyAPPetite	Manual data input, no AI predictions	Automates stage prediction with ML models
WebMD Symptom Checker	Generic, lacks personalization	Personalized chat and SHAP explainability per user

1.7 Vision Statement

For individuals and healthcare providers

Who need an accessible, intelligent, and early-warning system to monitor and predict chronic diseases such as Diabetes and Chronic Kidney Disease (CKD)

The AI Medical Assist

Is an AI-powered mobile health application

That predicts disease risk and stage using machine learning models, explains results through conversational AI (Gemini), and connects users to nearby medical professionals for timely care.

Unlike traditional symptom-checking or static health record apps,

Our product provides real-time predictive analytics, personalized AI explanations, and integrated doctor-location services — empowering users with data-driven, actionable, and explainable preventive healthcare.

1.8 System Limitations and Constraints

- **Limited Dataset Size:** The models depend on publicly available datasets; real clinical data integration is future work. Also we add stages level by ourselves. Here is the formula to generate the stage level .

An American Diabetes Association guideline states:

HbA1c < 5.7% → Normal.

HbA1c 5.7%–6.4% → Prediabetes

HbA1c ≥ 6.5% → Diabetes.

Likewise for fasting glucose:

s<100 mg/dL → Normal.

100–125 mg/dL → Prediabetes.

≥126 mg/dL → Diabetes.

1. **Internet Dependency:** Requires internet for Gemini API and Google Maps.
2. **Hardware Constraints:** Works best on Android devices with moderate resources.
3. **Non-clinical Certification:** The app provides educational insights only, not diagnostic authority.
4. **Server Resource Limitation:** Hosted locally via Docker Compose, not cloud-scaled.

1.9 Tools and Technologies

Table 1- 2Tools and Technologies for Proposed Project

Tools / Technologies	Version	Rationale
Python	3.11	Backend ML and API logic
Django REST Framework	3.15	Robust API layer and modular backend
Flutter	3.x	Cross-platform mobile development
PostgreSQL	15.x	Secure relational data storage
Redis	7.x	WebSocket pub/sub management
XGBoost	2.0	High-performance disease prediction
SHAP	0.44	Explainable AI for medical transparency
Google Maps API	Latest	Doctor location recommendations
Gemini API	Latest	Conversational health explanations

1.10 Project Deliverables

1. Source Code (Flutter + Django + ML)
2. Trained Models (ckd_model.pkl, diabetes_model.pkl)
3. PostgreSQL Database Schema
4. Documentation (SRS, Architecture, UML, API Docs)
5. Final Report (.docx and .pdf)
6. Presentation Slides (.pptx)
7. Docker Compose Environment for Local Execution

1.11 Project Planning

Phase	Timeline (Weeks)	Milestone
Requirement & Dataset Prep	1–2	Problem definition finalized
ML Model Development	3–4	Models trained and validated
Backend Development	5–6	Django REST APIs ready
Frontend UI Development	7–8	Flutter UI integrated
Chat + WebSocket Integration	9–10	Real-time AI chat operational
Maps Integration	11–12	Doctor lookup working
Testing & Evaluation	13–14	All modules validated
Documentation & Deployment	15–16	Project finalized & submitted

1.12 Summary

This chapter introduced the **AI Medical Assist** project, defined its problem domain, objectives, and architectural components. It demonstrated how AI and ML can assist in early disease detection and advisory healthcare. The project leverages **Django**, **Flutter**, and **Gemini AI** to deliver an explainable, secure, and scalable prototype that integrates both **predictive analytics** and **human-like conversational support**, setting the foundation for subsequent design and implementation phases.

Chapter 2

Requirements Analysis

2. Analysis

Provide an introduction to the chapter in a few lines. Write details in each of the sections provided ahead.

2.1 User classes and characteristics

Identify the various user classes that you anticipate will use this product and describe their pertinent characteristics using 2 column table.

2.2 Requirement Identifying Technique

This section describes the requirements identifying technique(s) which further help to derive functional requirements specification. The selection of the technique(s) will depend on the type of project. For instance,

- **Use case** includes **detailed use case descriptions + use case diagram**) is an effective technique for interactive end-user applications. Use case name and associated requirements must be presented here. However, use case descriptions must be given in fully dressed format.
- **Storyboarding** for graphically intensive applications. Visual representation of sequence of events, designs to represent the flow.

2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is typically organized by feature as a system feature name and specific functional requirements associated with this feature. It is just one possible way to arrange them. Other organizational options include arranging functional requirements by use case, process flow, mode of operation, user class, stimulus, and response depend on what kind of technique has been used to understand functional requirements. Hierarchical combinations of these elements are also possible, such as use cases within user classes.

2.3.1 Functional Requirement X

Itemize the specific functional requirements associated with each feature. These are the software capabilities that must be implemented for the user to carry out the feature's services or to perform a use case. Describe how the product should respond to anticipated error conditions and to invalid inputs and actions. Uniquely label each functional requirement, as described earlier. You can create multiple attributes for each functional requirement, such as rationale, source, dependencies, etc. The following template is required to write functional requirements.

Table 1- 1Description of FR-1

Identifier	FR-1
------------	------

Title	Title of requirement
Requirement	Description of requirement which may be written either from the user or system perspective e.g. If written in a user perspective The [user class or actor name] shall be able to [do something] [to some object] [qualifying conditions, response time, or quality statement]. If written in a system perspective [optional precondition] [optional trigger event] the system shall [expected system response]
Source	Where this requirement comes from (who originate it)
Rationale	The motivation behind the requirement
Business Rule (if required)	Any restriction, policy, the rule that the particular requirement must be fulfilled through its functional behaviour
Dependencies	Requirements ID that is dependent on this requirement
Priority	High/Medium/Low

2.4 Non-Functional Requirements

This section specifies nonfunctional requirements other than constraints, supporting requirements recorded in section 2.3, and external interface requirements. These quality requirements should be specific, quantitative, and verifiable. The following are some examples of documenting guidelines.

2.4.1 Reliability

Requirements about how often the software fails. The measurement is often expressed in MTBF (mean time between failures). Be sure to specify the consequences of software failure, how to protect from failure, a strategy for error detection, and a strategy for correction.

2.4.1 Usability

Usability requirements deal with ease of learning, ease of use, error avoidance and recovery, the efficiency of interactions, and accessibility. The usability requirements specified here will help the user interface designer create the optimum user experience. It may be quantified using the concepts of effort required to perform a task in terms of number of interactions required, considerations and solutions to make an accessible solution.

2.4.2 Performance

State specific performance requirements for various system operations. It may include response time or expected resources utilization requirements. If different functional requirements or features have different performance requirements, it's appropriate to specify those performance goals right with the corresponding functional requirements, rather than collecting them in this section.

2.4.3 Security

One or more requirements about protection of your system and its data. The measurement can be expressed in a variety of ways (effort, skill level, and time) to break into the system. Do not discuss solutions (e.g. passwords) in a requirements document.

2.5 External Interface Requirements

This section provides information to ensure that the system will communicate properly with users and with external hardware or software elements. A complex system with multiple subcomponents should create a separate interface specification or system architecture specification. The interface documentation could incorporate material from other documents by reference. For instance, it could point to a hardware device manual that lists the error codes that the device could send to the software.

2.5.1 User Interfaces Requirements

Describe the logical characteristics of each user interface that the system needs. Some possible items to include are

- References to GUI standards or product family style guides that are to be followed.
- Standards for fonts, icons, button labels, images, colour schemes, field tabbing sequences, commonly used controls, and the like.
- Screen layout or resolution constraints.
- Standard buttons, functions, or navigation links that will appear on every screen, such as a help button.
- Shortcut keys.
- Message display conventions.
- Layout standards to facilitate software localization.
- Accommodations for visually impaired users.

Document the user interface design details, such as specific dialog box layouts, in a separate user interface specification, not in the SRS. Including screen mock-ups in the SRS to communicate another view of the requirements is helpful but make it clear that the mock-ups are not the committed screen designs. If the SRS is specifying an enhancement to an existing system, it sometimes makes sense to include screen displays exactly as they are to be implemented. The developers are already constrained by the current reality of the existing

system, so it's possible to know up front just what the modified, and perhaps the new, screens should look like.

2.5.2 Software interfaces

Describe the connections between this product and other software components (identified by name and version), including other applications, databases, operating systems, tools, libraries, websites, and integrated commercial components (If any).

2.5.3 Hardware interfaces

Describe the characteristics of each interface between the software components and hardware components, if any, of the system. This description might include the supported device types, the data and control interactions between the software and the hardware, and the communication protocols to be used.

2.5.4 Communications interfaces

State the requirements for any communication functions the product will use, including email, web browser, network protocols, and electronic forms.

2.6 Summary

Summary of the processes involved in requirements identification and uses case analysis. It may also reiterate requirements engineering process's importance in addressing the projects objectives.

Chapter 3

Design and Architecture

3. System Design

Explain the product perspective in which software will operate including dependencies and interaction with other system. List the design constraints (like performance requirements) impacting design.

3.1 Design considerations

Assumptions and Dependencies: Lists any assumptions and dependencies that influence the design decisions.

Limitations: Describes any limitations in the software that could affect its functionality, performance, or compatibility.

Risks: Identifies potential risks and how they were addressed in the design

3.2 Design Models

Provide the descriptions of the following models used to describe the system design. Also ensure visibility of all diagrams.

The applicable models for the project using object-oriented development approach may include:

1. Class Diagram
2. Interaction Diagram (Either sequence or collaboration)
3. State Transition Diagram (for the projects which include event handling and backend processes)

3.3 Architectural Design

Develop a component structure and explain the relationships between these components to achieve the complete functionality of the system. This is a high-level overview of system's inter-modules collaboration to achieve the desired functionality.

Provide a diagram showing the major subsystems and their connections.

- UML Class relationship diagram and UML Component diagram
- After finalizing architecture style/pattern diagram (MVC, Client-Server, Layered, Multitiered) create a detailed mapping modules/components to each part of the architecture

3.4 Data Design

Explain how the information domain of your system is transformed into data structures. Describe how the major data or system entities are stored, processed, and organized. This includes the dataset/ database design and/or other design of other data structures used.

3.4.1 Data Dictionary

- **Alphabetical List of System Entities or Major Data with Types and Descriptions:**
 - Prepare an alphabetical list of all system entities or significant data. For each entry, include the **data type** and provide a brief **description** of its purpose within the system.
 - Present this information in a **two-column format**:
 - The **first column** will contain the **terminology** (e.g., object name, attribute, method, or method parameter).
 - The **second column** will provide the **description**, including data types, roles, or any other relevant information
- **Structured Approach: Functions and Function Parameters:**
 - For systems following a structured programming paradigm, list **all functions** along with their **function parameters**. For each function, provide a description detailing its functionality, and for each parameter, specify the **data type** and its role within the function.
- **Object-Oriented (OO) Approach: Objects, Attributes, Methods, & Method Parameters:**
 - For systems using the Object-Oriented (OO) approach, list the **objects** and their **attributes**, followed by a description. Additionally, list the **methods** associated with each object, along with their **method parameters**.

3.5 User Interface Design

Describe overview of the functionality of the system from the user's perspective. Explain how the user will be able to use your system to complete all the expected features and the feedback information that will be displayed for the user. [See the User Interface Design in Appendix -]

3.3.1 Screen Images

Display few screenshots showing the interface from the user's perspective. These can be handdrawn, or you can use Prototyping tool like Canva/ Figma. Just make them as accurate as possible.

3.3.2 Screen Objects and Actions

A discussion of screen objects and actions associated with those object for two use cases.

3.4 Behavioural Model

Interaction Diagrams: Includes diagrams (Sequence or Collaboration diagram) that illustrate interactions among components. Make sure that objects & methods mentioned in the interaction diagram are consistent with the class diagram

State Diagrams: Shows state transitions within the software, especially useful for systems with complex workflows or lifecycle states.

Make sure that each diagram has assigned a proper caption, so that table of figures can be created.

3.5 Design Decisions

Highlight the design choices while designing your system (e.g. choosing the Object oriented design pattern, choosing between the algorithmic approaches, normalization level of the database etc). Clearly specify which approach you have used and why you have made a specific design choice.

3.6 Summary

Summary of the key design ideas discussed, design refinement and decisions taken in the chapter. It may also reiterate chapter's importance in addressing the projects objectives

Chapter 4

Implementation

4. Implementation

This chapter discusses implementation details of the project. **Do not** put your source code here, however, you are required to write the core components functionalities in pseudocode form (Following sections are required in this chapter).

Note: You are required to follow proper coding standard to write your source code. For guidelines, **General Coding Standards & Guidelines** are provided in Appendix D.

4.1 Algorithm

Mention the algorithm(s) and its complexity used in your project to get the work done with regards to major modules. Provide a pseudocode explanation regarding the functioning of the core features. Be sure to use the correct syntax and semantics for algorithm representations. Following are few examples of algorithms/pseudocode:

Table 5- 1 Example of Algorithm

Algorithm 1 MHCF component	
Input:	
Output:	
<pre>1: merge ← true 2: Flag ← false 3: While(merge=='true') do: 4: merge ← false 5: for i in range (0: len(G)-1): 6: for j in range (i+1: len(G)):</pre>	

4.2 External APIs/SDKs

Describe the third-party APIs/SDKs used in the project implementation in the following table. Few examples of APIs are provided in the table.

Table 5- 1 Details of APIs used in the project

Name of API and version	Description of API	Purpose of usage	List down the API endpoint/function/class in which it is used

4.3 Code Repository

In order to manage the version control and collaboration effectively for the group project, Git will be used as the primary tool. All project files, including code, documentation, and other relevant resources, will be stored in the Git repository. This will ensure proper tracking of

changes, collaboration among team members, and access to the most up-to-date version of the project.

Git Repository Link:

The repository for the group project can be accessed using the following link:

[Git repository link]

4.3.1 **Metrics of the Git Repository:** The following metrics will be monitored to ensure effective use of the Git repository:

Commits: The number of commits made to track the frequency and consistency of contributions by team members.

Branches: The number of active and merged branches, representing the organization of different features and stages of development.

Pull Requests: The number of pull requests created, merged, or in review, showing the collaboration and code review process.

Issues: The number of open and closed issues, indicating bug tracking, feature requests, or task management.

Contributors: A list of contributors and their contribution statistics (commits, lines added, and lines removed).

Code Reviews: The number of reviews conducted on pull requests to ensure code quality and compliance with project standards.

4.4 Summary

Summary of the key points discussed in the chapter. It may also reiterate chapter's importance in addressing the projects objectives

Chapter 5

Testing and Evaluation

5. Introduction

In order to ensure software verification and validation, software testing is conducted. Though it is preferred to specify automated test scripts, testcases referring to manual testing can also be presented. Test scripts must conform to the following best practices:

Keep Tests Independent: Ensure each test can run independently of others.

Use Meaningful Assertions: Make sure your assertions are clear and meaningful. **Handle**

Test Data: Use setup and teardown methods to manage test data.

Avoid Hardcoding: Use variables and configurations to avoid hardcoding values.

Make Tests Repeatable: Ensure that tests can be run multiple times without failure due to state persistence.

Follow the template (represented in Table-1) for documenting the testcases of each of these categories

5.1 Unit Testing (UT)

It's a level of software testing where individual units of a software/component are tested. The purpose is to validate that each unit of the software performs as designed.

5.2 Functional Testing (FT)

In this functional testing, the functionality of each of the module/ stakeholders' view is tested. This is to ensure that the system produced meets the specifications and requirements

5.3 Integration Testing (IT)

After ensuring the individual test runs successfully, the interaction of the classes/ functions is tested to assess whether the feature is yielding the desired results.

5.4 Performance Testing (PT)

In order to assess the performance of the feature, the performance test is conducted. It usually measures response time, stability (reliability under stress). Specify the tool used for conducting the performance testing. Specify the screen shots (2-4) representing the configuration as well as output of the performance testing conducted.

Table 1- Testcase

Testcase ID	Unique identifier of testcase (e.g. UT1 for Unit testcase1)
Requirement ID	Requirement ID of the requirement to be tested
Title	Feature name or requirement to be tested must be represented in brief
Description	Description of feature to be tested
Objective	Why this testcase is needed

Driver/precondition	Setup required before test execution
Test steps	Step by step instructions to test
Input	Input to be provided to the testcase
Expected Results	Expected outcome of the testcase
Actual Result	Actual outcome of the testcase
Remarks	Test Status (Fail/Pass)

5.5 Summary

Summary of the key points discussed in the chapter. It may also reiterate chapter's importance in addressing the projects objectives

Chapter 6

System Conversion

6.Introduction

System conversion is the process of changing from one information system to another. This involves transferring data, processes, and operations from an old system to a new one.

6.1 Conversion Method

Choose and specify the conversion method for your project. There are several methods for converting systems:

Direct Conversion:

This method involves abruptly stopping the old system and immediately starting the new one. It's risky but can be cost-effective.

Parallel Conversion:

Both the old and new systems operate simultaneously for a period, allowing for comparison and verification. This is less risky but more expensive **Pilot**

Conversion:

The new system is implemented in a limited part of the organization, allowing for testing and adjustments before full implementation.

Phased Conversion:

The new system is implemented in stages, allowing for gradual adjustments

6.2 Deployment

This may include building the application, deployment, restoring a database from its backup, configuration to put the application in operational usage. It also includes the steps to start the application and checking successful deployment.

It is expected to present the steps required to make the software operational on the customer site. If the project is not for real clients, students are expected to mention the steps required to deploy it on docker.

6.2.1 Data Conversion

Specify the steps used to convert the data from your system to the deployed system. These steps may include data extraction and cleaning from existing system, validating data and loading it into target system. Data backup and restore procedure is one of the examples of the system.

6.2.2 Training

Section specifies user manual to train the users in using the application. User manual of two main use cases of the system can be presented.

6.3 Post Deployment Testing

Steps required to assess that software is properly deployed and converted. This includes running the testcases to ensure correct conversion of system

6.4 Challenges

List down the challenges encountered & their solutions while deploying the system and putting it in the operational usage.

6.5 Summary

Present the summary of the deployment process. Mention the challenges/ issues encountered, also present the ways those issues were resolved.

Chapter 7

Conclusion

7. Introduction

This chapter concludes the project, evaluate the project objectives & goals and highlights future work.

7.1 Evaluation

Make a list of the objectives specified in Chapter 1 and trace whether they have been implemented in your developed FYDP.

Objectives	Status
Recommend course in at most 2minutes	Completed

7.2 Traceability Matrix

Shows how each design, code and test configuration item traces back to a specific requirement in the Software Requirements Specification (SRS), ensuring that all requirements are accounted for in the final product. Use table to document requirements traceability, where requirement ID is a unique identifier assigned to the requirement, requirement description refers to brief description of the requirement discussed. Design specification entails list of the components/classes/algorithm where the requirement is addressed. Code represents the name of the file where this functionality is developed. Test ID represent the ID of the testcase documenting test data and procedure.

Make sure that the list is complete and consistent with requirements mentioned in SRS

Requirement ID	Requirement Description	Design Specification	Code	Test ID
R1	Recommend Course to the user	Component "Recommend"	Course.aspx	UTI

7.3 Conclusion

Conclude the section with concise summary statement, reinforcing the significance of your project's contributions, its significance and the fulfilment of its objectives agreed in the beginning of the project. Provide the table specifying correlation of objectives defined and functionality provided by the system.

7.4 Future Work

Provide suggestions or recommendations for future work or enhancements to your project. This could include additional features, optimisations, scalability improvements, or other potential applications which were excluded from this project's scope.

References

List any documents or other resources to which this SRS refers, if any. These might include user interface style guides, standards, system requirements specifications, interface specifications, or the SRS for a related product. The following are a few examples of different resources.

Book

Author(s). Book title. Location: Publishing company, year, pp.

Example:

W.K. Chen. Linear Networks and Systems. Belmont, CA: Wadsworth, 1993, pp. 123-35.

Article in a Journal

Author(s). "Article title". Journal title, vol., pp, date.

Example:

G. Pevere. "Infrared Nation." The International Journal of Infrared Design, vol. 33, pp. 56-99, Jan. 1979.

Articles from Conference Proceedings (published) Author(s).

"Article title." Conference proceedings, year, pp.

Example:

D.B. Payne and H.G. Gunhold. "Digital sundials and broadband technology," in Proc. IOOC-ECOC, 1986, pp. 557-998.

World Wide Web

Author(s)*. "Title." Internet: complete URL, date updated* [date accessed].

M. Duncan. "Engineering Concepts on Ice. Internet: www.iceengg.edu/staff.html, Oct. 25, 2000 [Nov. 29, 2003].

AppendixA

Use case Description Template

Appendix-A Use Case Description(Fully Dressed Format)

The Table A-1 below indicates a comprehensive use case template

Table A- 1Show the detail use case template and example

Use Case ID:	Enter a unique numeric identifier for the Use Case. e.g. UC-1
Use Case Name:	Enter a short name for the Use Case using an active verb phrase.
Actors:	[An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks.]
Description:	[Provide a brief description of the reason for and outcome of this use case.]
Trigger:	[Identify the event that initiates the use case.]
Preconditions:	[List any activities that must take place, or any conditions that must be true, before the use case can be started.]
Postconditions:	[Describe the state of the system at the conclusion of the use case execution.]
Normal Flow:	[Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions.
Alternative Flows:	[Document legitimate branches from the main flow to handle special conditions (also known as extensions). For each alternative flow reference the branching step number of the normal flow and the condition which must be true for this extension to be executed]
Business Rules	Use cases and business rules are intertwined. Some business rules constrain which roles can perform all or parts of a use case. Perhaps only users who have certain privilege levels can perform specific alternative flows. That is, the rule might impose preconditions that the system must test before letting the user proceed. Business rules can influence specific steps in the normal flow by defining valid input values or dictating how computations are to be performed]
Assumptions:	[List any assumptions].

Appendix-B

Coding Standards

Appendix-B General Coding Standards & Guidelines

1. Follow a consistent variable naming convention throughout the code. E.g. o Snakecase (words are delimited by “_” like: variable_one) o Pascalcase (words are delimited by capital letters like: VariableOne) o Camelcase (words are delimited by capital letters except the initial word like: variableOne)
 - o Hungarian Notation (describes the variable type or purpose at the start of the variable name like: arrDistributeGroup)
2. Use naming that visually describes scope like privateField, Const etc
3. Use read only/immutable when a field’s value should not be changed after initialization
4. Use only get, for properties that should not be updated from outside
5. Name functions according to their functionalities.
6. Insert appropriate comments to make the code understandable to any reader. Additionally follow a consistent style to do so. E.g.

```
/* the below function will be used for the addition of two variables*/  
int Add(){  
    //logic of the function  
}
```

Avoid commenting on obvious things
7. Make use of indentation for indicating the start and end of the control structures along with a clear specification of where the code is between them.
8. Follow consistent naming convention for files and folders.
9. Follow proper structure for classes
10. Group code entities logically into projects/packages/modules/folders
 - a. Separate logical layers of application into different modules/services/utilities etc.
 - b. User separate files for each class, struct, interface, enum etc. Name of the file and the enclosing entity must be same. E.g., class Employee in Employee.cs/Employee.java
11. Define and use everything within the minimum scope possible
12. Use proper access modifier for all code entities if required
13. Code entities should have maximum cohesion and least coupling possible.
14. Follow DRY law.
 - a. Do not repeat code.
 - b. A piece of knowledge should exist only in one place within the codebase/application
 - c. Reuse code as much as possible
 - d. Always write short methods
 - e. Single method should not have too many logic, long conditional flow or too many parameters
15. Strictly follow Single Responsibility Principle (SRP) when writing methods, classes, modules, projects, packages, or any other code entities.
16. Write classes and other code entities that are easy to extend without modification.

17. Handle exceptions
18. Log exception and other significant event details
19. Follow a consistent convention for logging all over the application

Appendix C

Prototype

Appendix-C Application Prototype